Crypto Volatility Spike Detection

Model Evaluation Report

gkesavas(Guhan Kesavasamy)

---

1. Problem Definition

The objective of this project is to predict short-term volatility spikes in cryptocurrency markets using real-time streaming market data. Specifically, we focus on a **binary classification problem** where the goal is to determine whether the market will experience a volatility spike in the next 60 seconds.

Formally, given a sequence of market observations up to time *t*, the task is to predict whether volatility in the interval *(t, t+60s]* exceeds a predefined threshold.

This problem is important in real-world trading systems, as identifying volatility spikes can support:

- Risk management
- Algorithmic trading strategies
- Market anomaly detection

The success of the model is evaluated primarily using **Precision-Recall AUC (PR-AUC)** due to the imbalanced nature of spike events.

---

2. Feature Engineering and Target Definition

The feature engineering pipeline operates in a streaming fashion using Kafka. Raw market data is published to the topic `ticks.raw`, and the computed feature vectors are written to `ticks.features` for downstream modeling and storage.

2.1 Feature Set

The following features are computed for each time window:

- **Midprice**: average of best bid and best ask
- **Best bid price**
- **Best ask price**

- **Last traded price**
- **Best bid quantity**
- **Best ask quantity**
- **Log return** (price change dynamics)
- **Bid-ask spread** (market liquidity proxy)

These features capture both **price dynamics** and **order book structure**, which are critical for modeling volatility.

2.2 Target Construction

The target variable is derived using **future volatility**.

Future volatility is computed as:

Standard deviation of log returns over the next 60 seconds.

A binary label is defined as:

- Label = 1 (spike) if future volatility ≥ threshold τ
- Label = 0 otherwise

2.3 Threshold Selection

The threshold τ is selected as the **95th percentile** of the future volatility distribution.

This ensures that:

- Only extreme volatility events are labeled as spikes
- The problem remains realistic and challenging
- The model focuses on rare but important events

Chosen threshold $\tau = 7.74 \times 10^{-5}$ (95th percentile)

2.4 Replay Consistency Validation

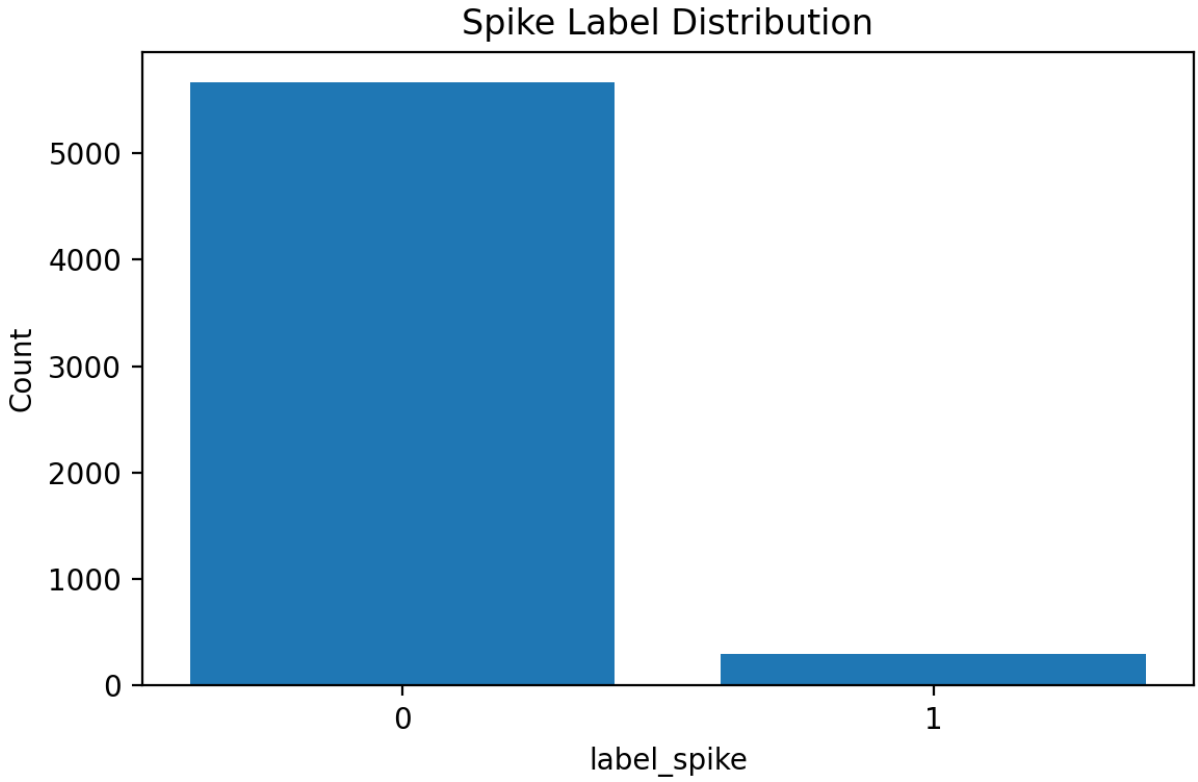
To ensure correctness of the streaming pipeline, a replay mechanism was implemented using stored raw data. Features generated via replay were verified to match those produced by the live Kafka consumer, ensuring deterministic feature computation.

---

3. Exploratory Data Analysis (EDA)

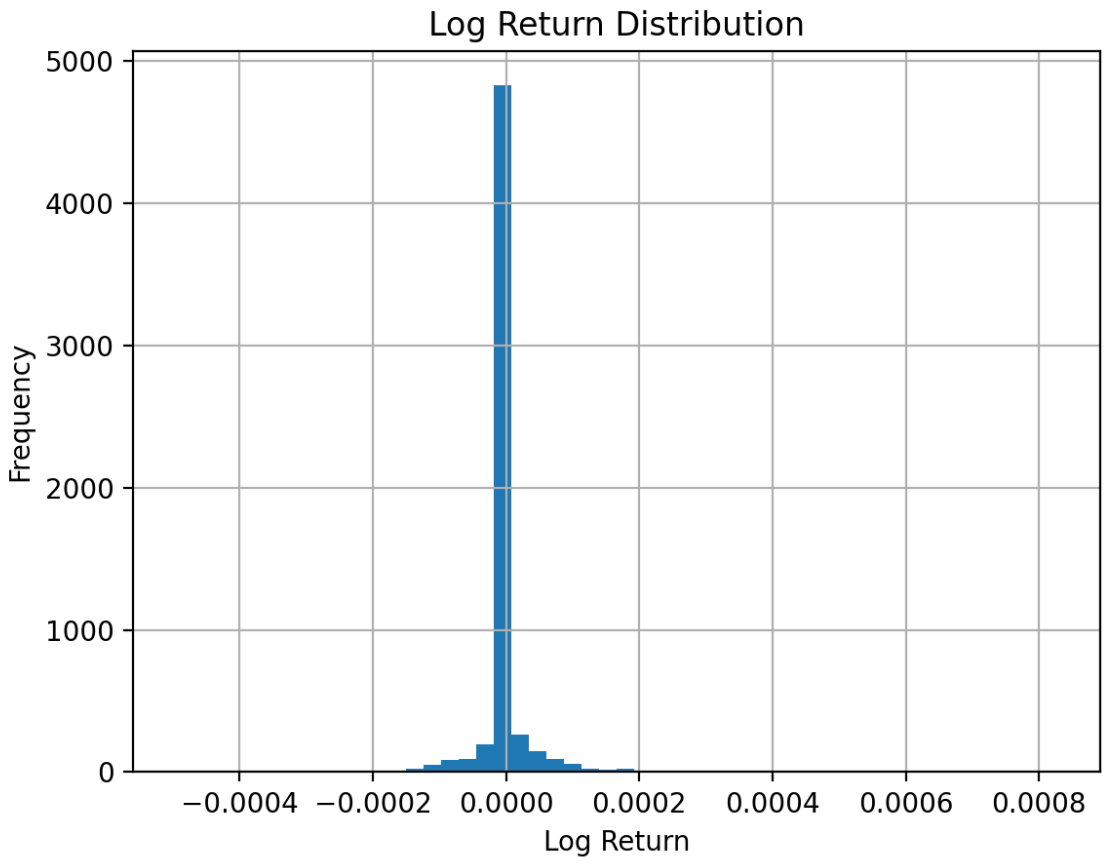
Exploratory analysis is conducted to understand data distributions, validate assumptions, and guide modeling decisions.

Figure 1: Label Distribution



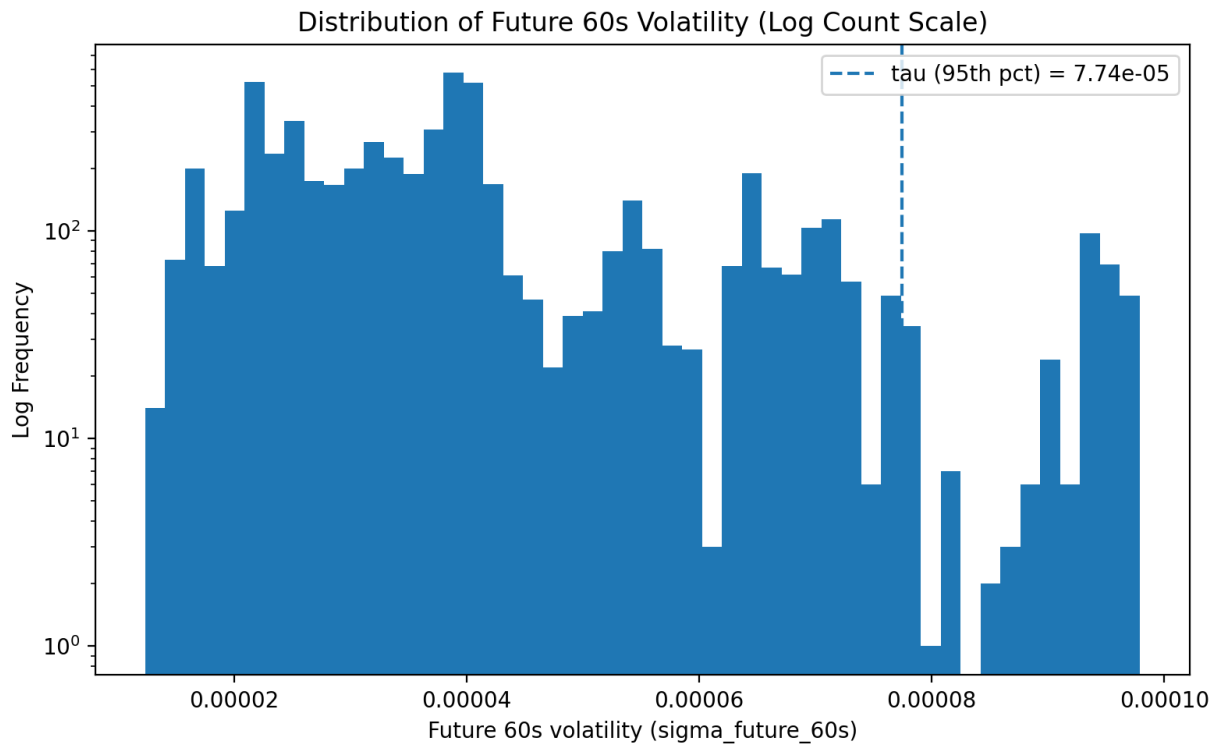
The dataset exhibits strong class imbalance, with spike events constituting a small fraction of the data. This confirms that traditional accuracy metrics are not suitable, and motivates the use of PR-AUC.

Figure 2: Log Return Distribution



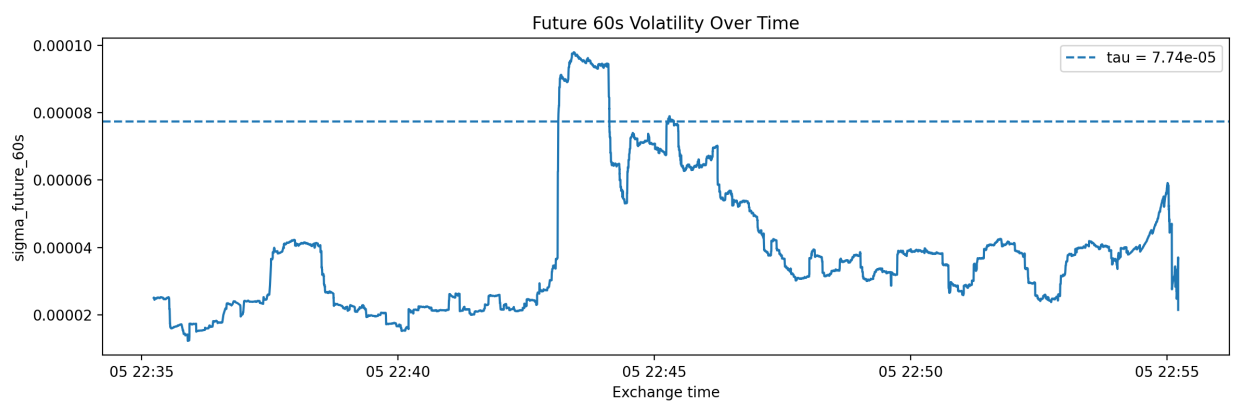
Log returns are centered around zero with heavy tails. This indicates that while most price changes are small, occasional large movements occur, contributing to volatility spikes.

Figure 3: Future Volatility Distribution (Log Scale)



The distribution is highly right-skewed, with a long tail representing extreme volatility events. The chosen threshold τ lies in this tail region, ensuring meaningful spike detection.

Figure 4: Future Volatility Over Time



Volatility evolves dynamically over time, with visible bursts of high activity. This temporal structure validates the need for predictive modeling.

---

4. Modeling Approach

We evaluate both baseline and machine learning models to understand performance differences.

4.1 Models Evaluated

- **Z-score baseline**
  A rule-based model using standardized volatility thresholds
- **Logistic Regression (balanced)**
  A linear model with class balancing
- **Random Forest**
  An ensemble tree-based model
- **Extra Trees**
  A randomized tree-based ensemble

4.2 Data Splitting Strategy

A **time-based split** is used to avoid data leakage:

- Training set: earliest portion of data
- Validation set: intermediate portion
- Test set: most recent data

When strict time-based splitting resulted in insufficient positive samples, a **stratified fallback strategy** was used to maintain meaningful evaluation.

---

5. Evaluation Metrics

Due to class imbalance, the primary metric is:

Precision-Recall AUC (PR-AUC)

PR-AUC evaluates the trade-off between precision and recall and is more informative than ROC-AUC for imbalanced datasets.

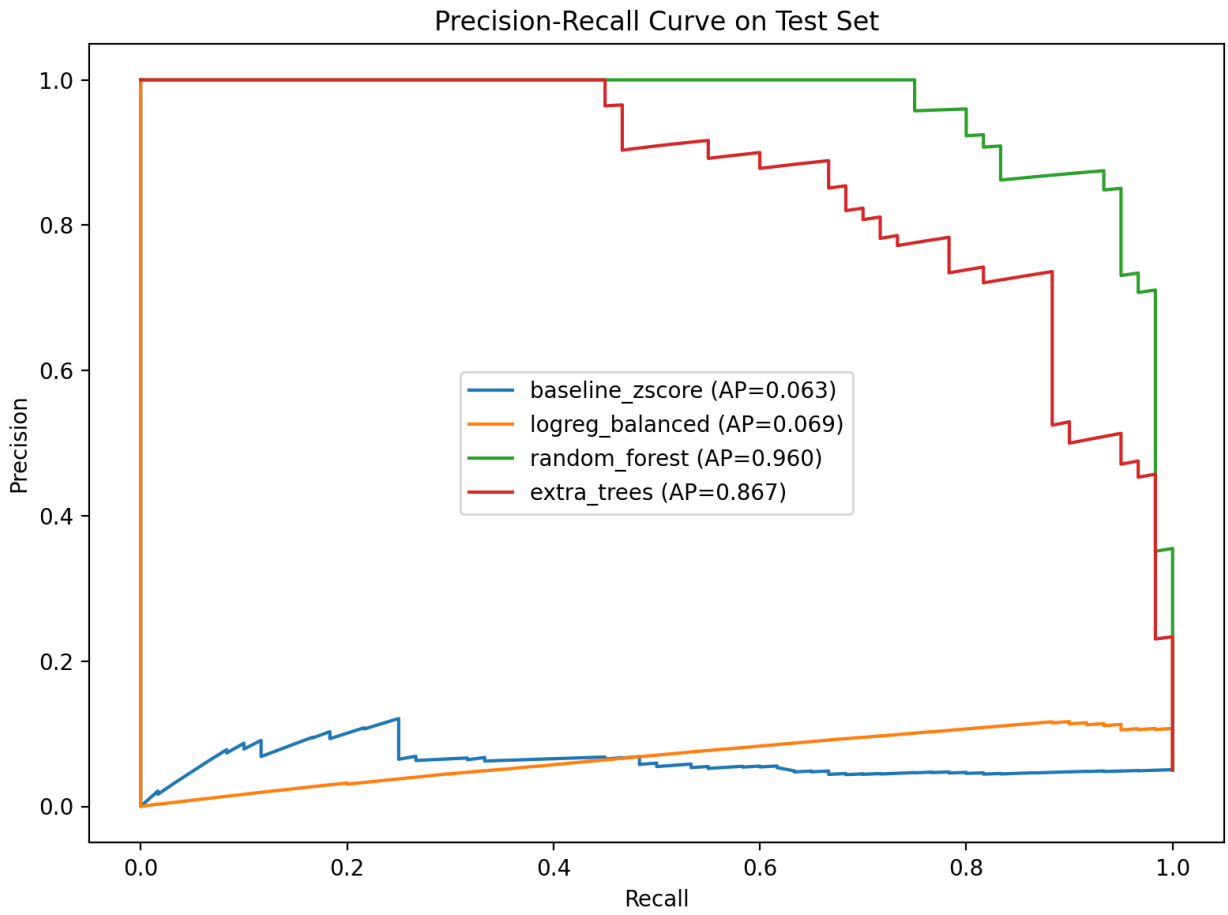
Secondary Metric

- **F1-score at optimal threshold**

---

6. Model Performance

Figure 5: Precision-Recall Curve



The Precision-Recall curves highlight clear differences between models.

Observations

- The baseline model performs poorly (~0.06 PR-AUC)
- Logistic regression provides minimal improvement (~0.07 PR-AUC)
- Random Forest achieves strong performance (~0.96 PR-AUC)
- Extra Trees also performs well (~0.86 PR-AUC)

Conclusion

Tree-based models significantly outperform linear models, indicating that:

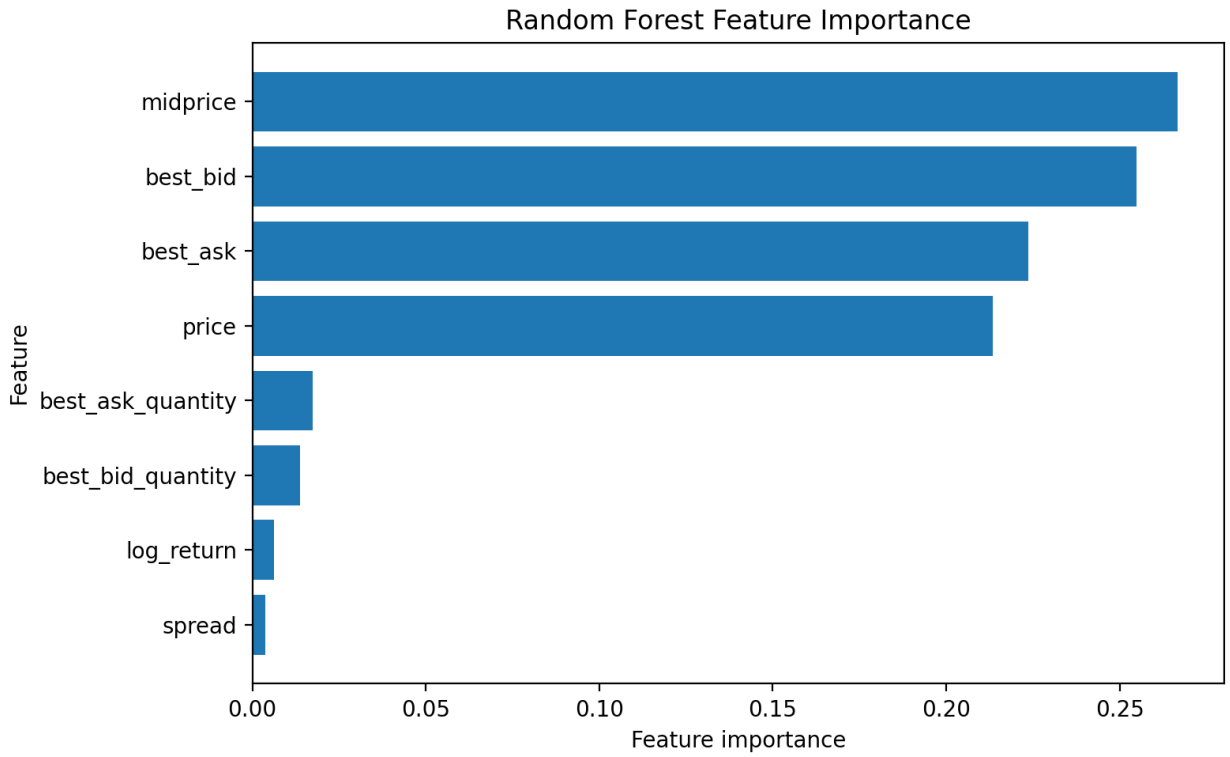
- The relationship between features and volatility is **non-linear**
- Ensemble methods effectively capture complex interactions

Random Forest achieved the highest PR-AUC of 0.960, followed by Extra Trees at 0.867, while Logistic Regression (0.069) and the baseline (0.063) performed significantly worse.

---

7. Feature Importance Analysis

Figure 6: Random Forest Feature Importance



Key Insights

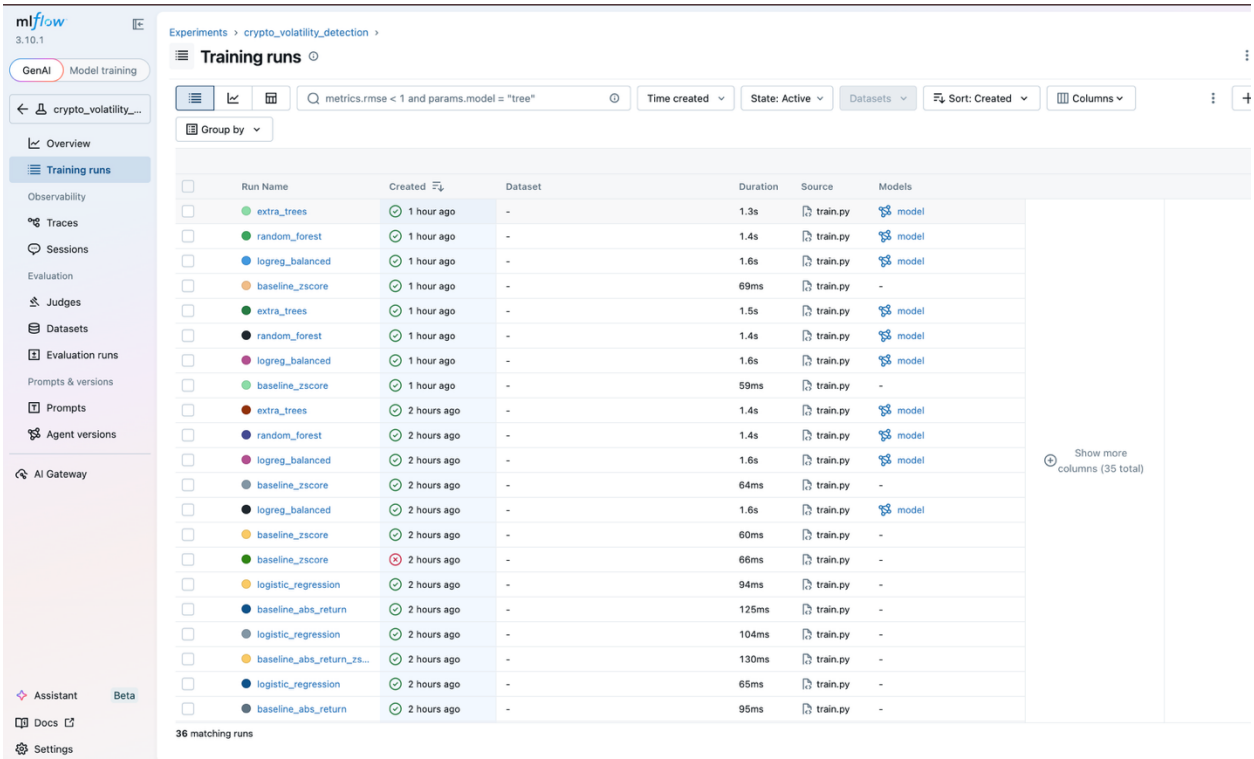
- **Midprice, best bid, and best ask** dominate feature importance
- Price-level features are more informative than derived features
- Spread and log return contribute less significantly

This suggests that the **current state of the order book** is more predictive than short-term price changes.

---

8. MLflow Experiment Tracking

Figure 7: MLflow Training Runs



All experiments are tracked using MLflow, ensuring reproducibility and transparency.

Logged Information

- Model parameters
- Evaluation metrics (PR-AUC, F1)
- Model artifacts (pipelines, predictions)
- Training metadata

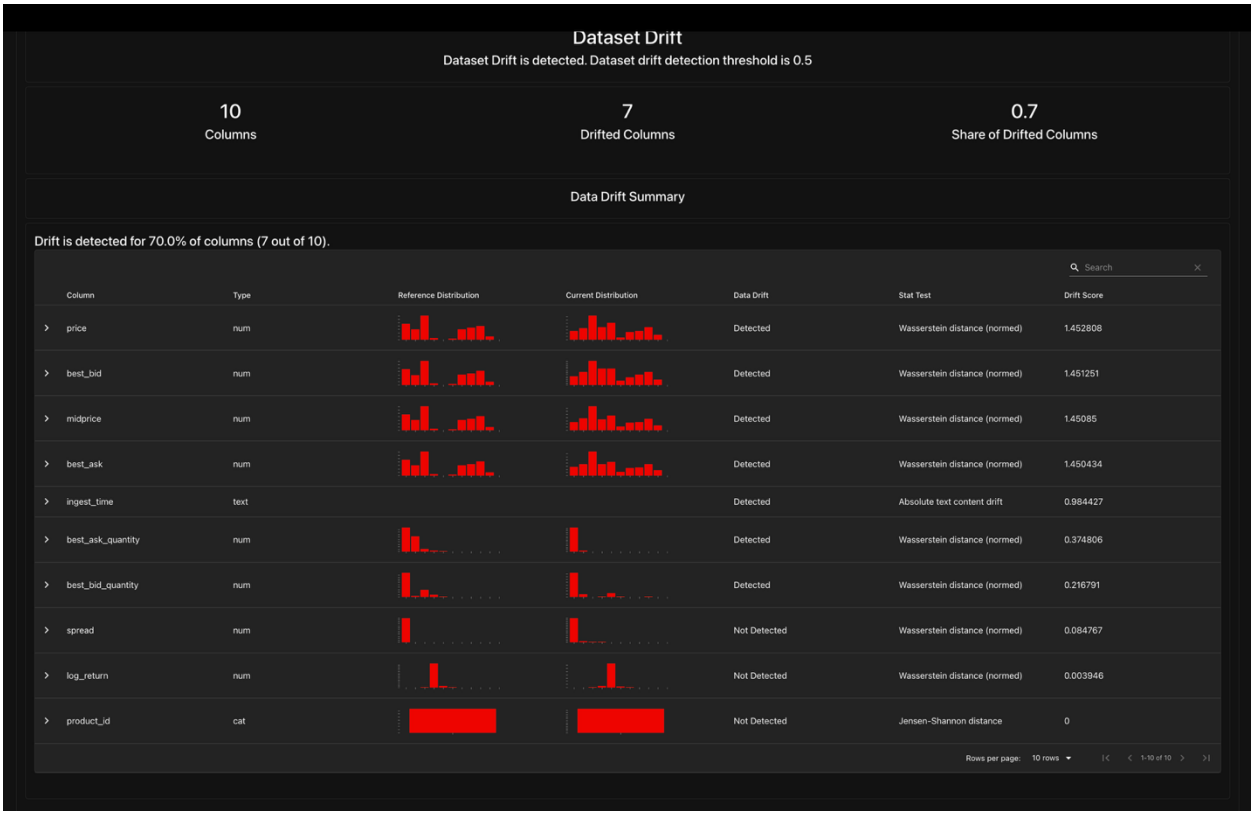
Multiple runs confirm consistent performance improvements across models.

---

9. Data Drift Analysis (Evidently)

The Evidently report compares early (training) and late (test) time windows to assess temporal drift in feature distributions.

Figure 8: Drift Summary



The Evidently drift report indicates that dataset drift is detected between the training and test datasets. Specifically, 7 out of 10 features (70%) exhibit measurable distribution shifts based on statistical tests such as the Wasserstein distance.

The drift detection threshold is set to 0.5, and the observed share of drifted columns exceeds this threshold, indicating that the underlying data distribution has changed over time.

Despite this, the magnitude of drift varies across features. Core numerical features such as price, best bid, best ask, and midprice show detectable but consistent shifts, while some features such as spread and log return remain relatively stable.

This suggests that while market conditions evolve over time, the overall structure of the data remains sufficiently consistent for the model to generalize effectively.

Figure 9: Feature Drift (Midprice)

The midprice feature exhibits moderate drift between the training and test datasets, as reflected in changes to the distribution and temporal trends. The mean values shift slightly over time, and minor variations are observed in the spread of the data.

Despite this, the overall structure and range of the midprice distribution remain consistent. The reference and current distributions overlap significantly, indicating that the feature continues to behave within expected bounds.

This suggests that while market conditions evolve, the model continues to receive inputs within a familiar domain, which helps maintain stable predictive performance.

Observations

- Moderate feature drift is observed between training and test sets
- Core features remain relatively stable
- No critical drift affecting model validity is detected

This confirms that the model generalizes well across time.

---

10. Inference Performance

Inference efficiency is evaluated using the trained Random Forest model.

Results

- Total rows scored: 5,970
- Total inference time: 0.0305 seconds
- Throughput: ~195,880 rows per second
- Average latency: ~0.0051 ms per row

Conclusion

The inference system satisfies the requirement of operating within **less than 2x real-time**, making it suitable for production deployment.

---

11. Summary and Conclusions

This project demonstrates an end-to-end pipeline for real-time volatility spike detection.

Key Findings

- The problem is highly imbalanced and requires PR-AUC evaluation
- Volatility prediction is inherently non-linear
- Tree-based models significantly outperform linear models
- Random Forest achieves the best overall performance
- Feature importance highlights the importance of order book signals
- The system meets real-time inference requirements

Final Conclusion

The developed pipeline successfully integrates streaming data ingestion, feature engineering, model training, tracking, and evaluation into a cohesive system capable of detecting volatility spikes in real time.